

Malware Concepts: Virus

With Trojan and Worm both covered, a virus sits precisely between them on the self-replication spectrum — and getting that positioning exactly right is the whole point of this guide, since "virus" is also the term most casually misused as a catch-all for malware in general. A computer virus, in the technically precise sense this guide uses, has one defining requirement neither of the previous two guides shares: **it needs a host file to attach itself to, and it needs a user to execute that host in order to spread.**

1. What a Virus Actually Is — The Foundation

A virus is malicious code that **attaches itself to a legitimate host file or program**, and replicates by inserting copies of itself into other files — but critically, unlike a worm, a virus cannot spread across a network on its own. It requires the infected host file to actually be **run or opened by a user** for the infection to transfer to a new system or new file.

The single defining trait: REQUIRES A HOST FILE + USER ACTION TO SPREAD.

A virus is not a standalone program the way a worm or Trojan typically is — it is code that **INFECTS an otherwise legitimate file**, and relies on normal, expected user behavior (running a program, opening a document, booting from infected media) to propagate that infection further.

Positioning Against the Other Two Guides

```
Trojan → standalone, disguised program; NO self-replication at all

Virus  → NOT standalone – attaches to a host file; self-replicates
        but needs a HOST + USER ACTION to spread to new
        files/systems

Worm   → standalone, self-replicating; spreads with NO user
        action needed at all, across a network autonomously
```

2. The Infection Cycle

Step-by-Step

Step	Stage	Description
1	ATTACHMENT	The virus inserts its own code into a legitimate host file (an executable, a document with macro capability, a boot sector). The host file often continues to appear and even function normally.
2	DORMANCY	Many viruses remain inactive until a specific trigger condition is met (a certain date, a certain number of executions, a specific user action). This dormant period is a deliberate design choice to delay detection and maximize spread before the payload activates.
3	EXECUTION	The infected host file is run by a user. This is the critical dependency — the virus cannot do this itself.
4	REPLICATION	Upon execution, the virus code runs alongside (or instead of) the host's normal function, and seeks out OTHER files on the same system to infect, continuing the attachment cycle locally.
5	FURTHER SPREAD	The newly infected files spread to new systems only through NORMAL FILE SHARING/TRANSFER behavior — copying a file to a USB drive, emailing an infected document, sharing an infected program — never through the virus's own autonomous network activity.

Infection Flow

```
Attach to host file
  ↓
(Often dormant)
  ↓
User executes host
  ↓
Replicate to other local files
  ↓
Spread through normal file sharing
  ↓
New system / new file
  ↓
User executes
  ↓
Repeat
```

3. Categories of Virus by Infection Target

Virus Category	Description
File Infector Virus	Attaches to executable program files directly; activates when the infected program is run.
Macro Virus	Embeds itself in the macro/scripting capability of productivity software (spreadsheet and document macros being the classic historical example). It activates when the document is opened and macros are allowed to run.
Boot Sector Virus	Infects the boot sector of a storage device, activating before the operating system itself even fully loads. Historically spread via infected floppy disks, and is the conceptual ancestor of modern bootkit techniques referenced in the Rootkit guide.
Polymorphic Virus	Deliberately changes/mutates its own code signature with each new infection, specifically to evade static signature-based antivirus detection while preserving its underlying malicious function.
Metamorphic Virus	Goes even further than polymorphic variants, completely rewriting its own code structure (not just an encryption layer/signature) between infections, making signature-based detection dramatically harder still.

4. Why the "Requires a Host + User Action" Distinction Matters Practically

This isn't just taxonomic pedantry — it directly shapes both how a virus spreads in practice and how it's best defended against:

- A virus's spread rate is fundamentally bounded by **NORMAL HUMAN FILE-SHARING BEHAVIOR**, making it inherently slower and less explosive than a worm's autonomous, network-driven propagation.
- Because a virus depends on an existing legitimate host file, **file integrity monitoring** (detecting unexpected changes to known files) is a uniquely effective detection method against this category specifically, in a way that doesn't apply the same way to standalone Trojans or worms.
- Removing the **INFECTED FILE itself**, or the specific malicious code injected into it, is often sufficient to fully remediate a virus infection on a single system — there's no separate standalone malicious process to hunt down beyond the infected host file(s) themselves.

5. Real-World Impact

Impact	Description
Data and file corruption	Many virus payloads directly damage or corrupt the files they infect, sometimes as a deliberate destructive goal rather than a side effect of the infection mechanism itself.
Historical large-scale disruption	Macro viruses in particular caused widespread, organization-level disruption in the era before macro execution was disabled by default in productivity software.
Persistent reinfection risk	Because a virus can spread through ordinary file-sharing behavior, incomplete remediation (missing even one infected file during cleanup) can lead to reinfection well after the initial incident appears resolved.

6. Detection Considerations

- **File integrity monitoring / checksum comparison** against known-good file hashes is uniquely well-suited to virus detection, given the "attaches to an existing host file" mechanism described in Section 4.
- Unexpected changes in **file size, modification date, or behavior** of a previously-trusted, unchanged program are classic virus-infection indicators.
- Signature-based antivirus remains reasonably effective against **non-polymorphic variants**, but requires supplementing with heuristic/behavioral analysis specifically to catch the polymorphic and metamorphic variants described in Section 3.
- Disabling **macro auto-execution by default** in productivity software (now standard practice in modern versions) remains a meaningful detection/prevention layer against the macro virus category specifically.

7. Prevention

- Maintain updated antivirus/endpoint protection with both **signature-based and heuristic/behavioral detection capability**, given the polymorphic/metamorphic evasion techniques covered above.
- Disable **macro auto-execution** in productivity software by default, only enabling macros for specifically trusted, verified documents.

- Practice careful **file-sharing hygiene** — scan removable media and externally-received files before execution, given that ordinary file transfer IS the entire spread mechanism for this category.
- Maintain regular, **verified backups** specifically to support recovery from destructive virus payloads without needing to pay any form of ransom or accept permanent data loss.
- Apply the principle of **least privilege** to limit how far an infection can spread even after a single successful host-file infection on one system.

Quick Reference Cheat Sheet

Defining Trait

Attaches to a HOST FILE; requires USER ACTION (running/opening the infected file) to spread — cannot self-propagate over a network autonomously.

Infection Cycle

```
Attach to host file
  ↓
  (Often dormant)
  ↓
User executes host
  ↓
Replicate to other local files
  ↓
Spread to new systems only through
NORMAL file sharing
  ↓
Repeat
```

Categories by Infection Target

Category	Infection Target / Characteristic
File Infector	Executable program files
Macro Virus	Document/spreadsheet macro capability
Boot Sector Virus	Storage device boot sector

Polymorphic Virus	Mutates signature per infection (evasion)
Metamorphic Virus	Fully rewrites its own code per infection (stronger evasion than polymorphic)

Key Distinction From Trojan / Worm

Malware Type	Defining Behavior
Trojan	Standalone; no self-replication; needs deception to run
Virus	Needs a HOST FILE + user action to spread
Worm	Standalone; self-replicates AND self-spreads with NO user action needed